1. What is a named, independent block of code designed to perform a specific task called?

(A) Function
(B) Variable
(C) Structure
(D) Macro

2. Which statement in C informs the compiler about a function's name, return type, and parameters?

(A) Function prototype
(B) Function definition
(C) Function call
(D) Header file

3. What is the return type of a function that does not return any value to the caller?

(A) void
(B) null
(C) empty
(D) int

4. What are the arguments passed into a function call called?

(A) Actual arguments
(B) Formal parameters
(C) Dummy parameters
(D) Global arguments

5. What are the variables declared in the function definition header called?

(A) Formal parameters
(B) Actual parameters
(C) External variables
(D) Register variables

6. In C, standard function parameter passing is performed by default using:

(A) Call-by-value
(B) Call-by-reference
(C) Call-by-name
(D) Call-by-result

7. What happens to the caller's variable when passed into a function by value?

(A) Remains unmodified
(B) Is permanently updated
(C) Memory is freed
(D) Throws error

8. How can call-by-reference semantics be achieved in ANSI C?

(A) Passing pointers (&x)
(B) Using reference & in C++
(C) Using global arrays
(D) Using register keyword

9. What is a function that calls itself directly or indirectly called?

(A) Recursive function
(B) Inline function
(C) Nested function
(D) Static function

10. Which mandatory condition in a recursive function terminates the recursion?

(A) Base case
(B) Recursive step
(C) Infinite case
(D) Default case

11. What runtime error occurs when recursion lacks a valid base case or recurses too deeply?

(A) Stack overflow
(B) Heap exhaustion
(C) Compilation error
(D) Syntax fault

12. Where are local function variables and activation records stored at runtime?

(A) Call stack
(B) Heap
(C) BSS segment
(D) Data segment

13. How many storage classes are formally supported in the C programming language?

(A) 4 (auto, reg, stat, ext)
(B) 2
(C) 3
(D) 5

14. Which storage class is the default for local variables declared inside a function block?

(A) auto
(B) static
(C) extern
(D) register

15. What initial value does an uninitialized auto local variable contain in C?

(A) Garbage / indeterminate
(B) 0 (zero)
(C) 1
(D) NULL

16. Which storage class requests that a variable be stored in a high-speed CPU register?

(A) register
(B) auto
(C) static
(D) volatile

17. Which unary operator CANNOT be applied to a variable declared with register storage class?

(A) & (Address-of)
(B) * (Dereference)
(C) sizeof
(D) ++ (Increment)

18. Which storage class preserves variable value across multiple function invocations?

(A) static
(B) auto
(C) register
(D) volatile

19. To what value are uninitialized static and global variables automatically initialized?

(A) 0 (zero)
(B) Garbage
(C) -1
(D) 1

20. Where are static and global variables stored in the program's runtime memory layout?

(A) Data / BSS segment
(B) Call stack
(C) Heap
(D) Text segment

21. Which storage class declares a variable that is defined in another source file or module?

(A) extern
(B) auto
(C) static
(D) register

22. What linkage does a global variable or function marked with static have?

(A) Internal linkage
(B) External linkage
(C) No linkage
(D) Dynamic linkage

23. Can functions be defined inside other functions in standard ANSI C (nested functions)?

(A) No (strictly forbidden)
(B) Yes (fully supported)
(C) Only void functions
(D) Yes (since C99)

24. What keyword introduced in C99 suggests that the compiler replace a function call with its body?

(A) inline
(B) macro
(C) static
(D) extern

25. What header file must be included to implement functions accepting variable argument lists?

(A) <stdarg.h>
(B) <stdlib.h>
(C) <stdio.h>
(D) <varargs.h>

26. Which macro initializes a variable argument list (va_list) pointer in C?

(A) va_start
(B) va_arg
(C) va_end
(D) va_copy

27. Which macro retrieves the next argument of a specified type from va_list?

(A) va_arg
(B) va_start
(C) va_end
(D) va_next

28. Which macro cleans up and finalizes the variable argument list before return?

(A) va_end
(B) va_start
(C) va_arg
(D) va_free

29. What happens when an array name is passed as an argument to a function?

(A) Decays to pointer to 1st
(B) Entire array copied
(C) Throws error
(D) Passes array length

30. What is the scope of a variable declared inside a block enclosed by { }?

(A) Block / Local scope
(B) File scope
(C) Function prototype scope
(D) Global scope

31. What is the scope of a variable declared outside of all functions?

(A) File / Global scope
(B) Block scope
(C) Register scope
(D) Static scope

32. What is the lifetime of an automatic (auto) local variable?

(A) Duration of block
(B) Entire program run
(C) Infinite
(D) Until heap freed

33. What is the lifetime of a static local variable?

(A) Entire program run
(B) Duration of block
(C) Single function call
(D) Until return

34. Can a local variable have the exact same identifier name as a global variable in C?

(A) Yes (shadows global)
(B) No (compile error)
(C) Only if float
(D) Only if static

35. What does the return statement without an expression (return;) do?

(A) Exits void function
(B) Returns 0
(C) Returns NULL
(D) Throws error

36. In C, what is the default return type of a function if omitted in older C89?

(A) int
(B) void
(C) float
(D) char

37. Which recursion type occurs when the recursive call is the very last operation executed?

(A) Tail recursion
(B) Head recursion
(C) Tree recursion
(D) Indirect recursion

38. Why is tail recursion favored by optimizing compilers?

(A) Can be optimized to loop
(B) Saves disk space
(C) Prevents compilation
(D) Eliminates variables

39. How many function activation records exist on the stack for factorial(5)?

(A) 6 (from 5 down to 0)
(B) 1
(C) 5
(D) 120

40. Which recursive problem moves disks between rods without placing larger on smaller?

(A) Tower of Hanoi
(B) Traveling Salesperson
(C) Knapsack Problem
(D) N-Queens

41. What is the output: void f() { static int x = 0; x++; printf("%d ", x); } main() { f(); f(); f(); }?

(A) 1 2 3
(B) 1 1 1
(C) 0 1 2
(D) 3 3 3

42. Why does static int x retain its value between invocations of f()?

(A) Stored in data segment
(B) Stored on CPU stack
(C) Reallocated on entry
(D) Saved in registers

43. What is the output: void f() { int x = 0; x++; printf("%d ", x); } main() { f(); f(); f(); }?

(A) 1 1 1
(B) 1 2 3
(C) 0 0 0
(D) Garbage

44. What is the result of applying sizeof to a formal array parameter: void f(int arr[10]) { sizeof(arr); }?

(A) sizeof(int*) [4 or 8]
(B) sizeof(int)*10 [40]
(C) 10
(D) Undefined

45. Why does sizeof(arr) return pointer size inside a function parameter list?

(A) Array decays to pointer
(B) Compiler bug
(C) Only first element read
(D) Array is truncated

46. What is the output: void swap(int a, int b) { int t = a; a = b; b = t; } main() { int x=5,y=10; swap(x,y); printf("%d %d", x, y); }?

(A) 5 10 (unswapped)
(B) 10 5
(C) 0 0
(D) Compilation error

47. Why were x and y not swapped in the previous program?

(A) Passed by value (copies)
(B) Missing return statement
(C) Integer overflow
(D) Stack corrupted

48. What must the swap function signature be to successfully swap variables in C?

(A) void swap(int *a, int *b)
(B) void swap(int &a, int &b)
(C) int swap(int a, int b)
(D) void swap(int a[], b[])

49. What happens if a function prototype is omitted before its call in C99/C11?

(A) Compile error/warning
(B) Defaults to int return
(C) Generates assembly
(D) Permitted silently

50. What is the linkage of a global function declared without any storage class specifier?

(A) External linkage
(B) Internal linkage
(C) No linkage
(D) Dynamic linkage

51. How does declaring static void helper() affect its visibility across multi-file projects?

(A) Restricted to this .c file
(B) Visible to all .c files
(C) Exported to DLL
(D) Inlined everywhere

52. What linker error occurs if two separate .c files declare int global_counter = 0;?

(A) Multiple definition error
(B) Unresolved external
(C) Stack overflow
(D) Type mismatch

53. How should a shared global variable be declared in a header file to prevent linker errors?

(A) extern int global_var;
(B) int global_var = 0;
(C) static int global_var;
(D) auto int global_var;

54. What does the statement extern int buffer_count; do?

(A) Declares without defining
(B) Allocates memory buffer
(C) Initializes to zero
(D) Creates local copy

55. What is the minimum number of moves required to solve the Tower of Hanoi for N disks?

(A) 2^N - 1
(B) 2*N - 1
(C) N^2
(D) N!

56. How many total moves are required to solve Tower of Hanoi for N = 4 disks?

(A) 15 (2^4 - 1)
(B) 16
(C) 8
(D) 24

57. What is the time complexity of the standard recursive Tower of Hanoi algorithm?

(A) O(2^N)
(B) O(N^2)
(C) O(N log N)
(D) O(N!)

58. What is the time complexity of naive recursive Fibonacci: fib(n) = fib(n-1) + fib(n-2)?

(A) O(2^N) [Golden ratio]
(B) O(N)
(C) O(N^2)
(D) O(log N)

59. What is the time complexity of recursive binary search on a sorted array of size N?

(A) O(log N)
(B) O(N)
(C) O(1)
(D) O(N log N)

60. What is the recurrence relation for Euclidean GCD algorithm: gcd(a, b)?

(A) gcd(b, a % b)
(B) gcd(a - b, b)
(C) gcd(a / b, b)
(D) gcd(b, a)

61. What is the time complexity of the Euclidean algorithm to find GCD of two numbers?

(A) O(log(min(a, b)))
(B) O(a + b)
(C) O(min(a, b))
(D) O(1)

62. What is the advantage of using inline functions over preprocessor #define macros?

(A) Full type safety checks
(B) Faster disk storage
(C) Zero binary code size
(D) Dynamic dispatch

63. Why might a compiler ignore an inline function specifier?

(A) Function is too complex
(B) Missing semicolon
(C) Invalid data types
(D) Header conflict

64. Which function in <stdlib.h> registers a function to be executed automatically upon exit?

(A) atexit()
(B) exit()
(C) abort()
(D) register_exit()

65. What is the maximum number of functions that can be registered with atexit() in ANSI C?

(A) At least 32
(B) Exactly 1
(C) Unlimited
(D) 16

66. In what order are functions registered with atexit() executed upon program termination?

(A) Reverse order (LIFO)
(B) Registration order
(C) Alphabetical order
(D) Random order

67. In bioinformatics, what does the Hamming distance between two equal-length DNA strands measure?

(A) Number of mismatched bases
(B) Length difference
(C) Molecular weight
(D) GC percentage

68. What is the Hamming distance between DNA strands 'GAGC' and 'GATC'?

(A) 1 (at position 3)
(B) 0
(C) 2
(D) 4

69. In converting DNA to mRNA transcription, which base replaces Thymine (T)?

(A) Uracil (U)
(B) Cytosine (C)
(C) Guanine (G)
(D) Adenine (A)

70. What is an activation record / stack frame composed of at runtime?

(A) Locals, params, ret-addr
(B) Heap pointers only
(C) Global symbol tables
(D) Code instructions

71. What happens to the stack pointer register (ESP/RSP) when a function returns in machine code?

(A) Restores caller frame
(B) Zeros out memory
(C) Increments by heap size
(D) Freezes instruction

72. Can a function return a pointer to its own automatic local variable?

(A) No (dangling pointer!)
(B) Yes (valid ANSI C)
(C) Only if int*
(D) Yes if const

73. Why is returning a pointer to an auto local variable a critical bug?

(A) Stack frame is destroyed
(B) Type mismatch
(C) Pointer is zeroed
(D) Linker error

74. How can a function safely return a pointer to data created within it?

(A) Allocate on heap / static
(B) Use auto local variable
(C) Cast to void*
(D) Use register storage

75. What does declaring const int *arr in a function prototype enforce?

(A) Array elements read-only
(B) Array size is constant
(C) Pointer is immutable
(D) Array is in ROM

76. What compiler optimization replaces a tail-recursive call with an unconditional jump loop?

(A) Tail Call Optimization
(B) Loop unrolling
(C) Dead code elimination
(D) Constant folding

77. Which function signature illustrates a tail-recursive factorial helper?

(A) fact_tail(n, acc * n)
(B) n * fact(n - 1)
(C) fact(fact(n-1))
(D) 1 + fact(n - 1)

78. Why is standard fact(n) = n * fact(n-1) NOT tail-recursive?

(A) Multiplication after call
(B) n is decremented
(C) Returns integer
(D) Stack frame omitted

79. In <stdarg.h>, what is the second argument required by va_start(ap, last_arg)?

(A) Last named parameter
(B) First variadic arg
(C) Argument count
(D) Format string pointer

80. Why can float types not be retrieved directly using va_arg(ap, float)?

(A) Promoted to double
(B) Promoted to int
(C) Undefined syntax
(D) Float is truncated

81. Why can char types not be retrieved directly using va_arg(ap, char)?

(A) Promoted to int
(B) Promoted to double
(C) Char is unsigned
(D) Compiler crash

82. What is mutual / indirect recursion?

(A) A calls B, and B calls A
(B) A calls itself 2 times
(C) A calls main()
(D) A calls external DLL

83. How does the C compiler handle forward references in mutual recursion?

(A) Use forward prototype
(B) Rearrange source lines
(C) Use extern storage
(D) Cannot be compiled

84. What is memoization in the context of recursive algorithms?

(A) Caching computed results
(B) Freeing stack frames
(C) Converting to assembly
(D) Allocating virtual RAM

85. By what factor does memoization reduce Fibonacci time complexity from naive recursion?
(A) From O(2^N) to O(N)
(B) From O(N^2) to O(log N)
(C) From O(N!) to O(N^2)
(D) From O(N) to O(1)

86. In a re-entrant function, what type of variables must be strictly avoided?
(A) Static and global variables
(B) Local auto variables
(C) Pointer parameters
(D) Constant literals

87. What makes a function thread-safe in concurrent C programming?
(A) No shared unsynced state
(B) Written in assembly
(C) Contains no loops
(D) Returns void only

88. Which standard library function is an example of a non-reentrant function due to internal static state?
(A) strtok()
(B) strlen()
(C) strcpy()
(D) printf()

89. Which POSIX thread-safe alternative replaces strtok() by taking an explicit context pointer?
(A) strtok_r()
(B) strtok_s()
(C) strtok_safe()
(D) strtok_t()

90. In C, can main() be called recursively within user code?
(A) Yes (legal in C89/C99)
(B) No (strictly forbidden)
(C) Only in C11
(D) Triggers syntax error

91. Unlike C, why is calling main() recursively strictly illegal in ISO C++?
(A) C++ standard forbids it
(B) C++ lacks call stack
(C) Name mangling issues
(D) Main has no return

92. What is a function pointer in C?
(A) Variable holding code addr
(B) Pointer to return value
(C) Dynamic function call
(D) Macro replacement

93. What is the syntax to declare a pointer fp to a function taking two ints and returning double?
(A) double (*fp)(int, int);
(B) double *fp(int, int);
(C) double fp*(int, int);
(D) (*double) fp(int, int);

94. What does double *fp(int, int); declare instead due to operator precedence?
(A) Function returning double*
(B) Function pointer
(C) Array of doubles
(D) Syntax error

95. Which standard library sorting function takes a function pointer comparator as a callback?
(A) qsort()
(B) sort()
(C) bsearch()
(D) mergesort()

96. What return values must a qsort comparator callback return for elements a and b?
(A) <0 (a<b), 0 (=), >0 (a>b)
(B) Boolean true/false
(C) Always 0
(D) Difference in bytes

97. In calculating Hamming distance recursively, what is the base case?
(A) End of string ('\0')
(B) First mismatch
(C) String length == 10
(D) Null pointer only

98. What does the C99 restrict qualifier applied to a pointer parameter inform the compiler?
(A) No other pointer aliases it
(B) Pointer is read-only
(C) Pointer is restricted to 0
(D) Pointer is stack-only

99. What optimization can a compiler perform when pointers are marked with restrict?

(A) Aggressive caching in reg
(B) Disable all loops
(C) Inlines automatically
(D) Forces heap allocation

100. In multi-file builds, what happens if an inline function is defined in a header without static?

(A) Linker multiple definition
(B) Compiles cleanly
(C) Treated as extern auto
(D) Ignored by linker

SECTION B: DESCRIPTIVE & SUBJECTIVE QUESTIONS

Q1. Explain the principles of Modular Programming in C, detailing functional decomposition, interface abstraction, and code reusability.

Q2. Detail the anatomy of a C function, explaining function prototypes, definitions, calling conventions, and return mechanics.

Q3. Compare Call-by-Value with Call-by-Reference in C, illustrating stack frame isolation and pointer-based parameter modification.

Q4. Explain array parameter passing in C, detailing array decay to pointers and why array length cannot be determined via sizeof inside functions.

Q5. Explain recursion in C, detailing base cases, recursive steps, call stack mechanics, and stack overflow conditions.

Q6. Write an ANSI C program to solve the Tower of Hanoi problem recursively, detailing mathematical complexity and move proofs.

Q7. Explain Tail Recursion vs Non-Tail Recursion in C, detailing Tail Call Optimization (TCO) and stack frame recycling.

Q8. Detail the four C Storage Classes (auto, register, static, extern), comparing storage duration, scope, and memory segment allocation.

Q9. Explain local static variables vs global static variables vs global static functions in multi-file C architecture.

Q10. Detail external linkage using extern across multiple source files with an annotated multi-module C project layout.

Q11. Explain inline functions in C99, comparing them with preprocessor macros and detailing compiler inlining heuristics.

Q12. Detail variadic functions in C using <stdarg.h>, implementing a custom formatting print function my_printf().

Q13. Write an ANSI C program to compute the Hamming Distance between two DNA oligonucleotides using an iterative function.

Q14. Write an ANSI C program implementing a modular function to transcribe a DNA coding sequence into messenger RNA (mRNA).

Q15. Write an ANSI C program to compute the molecular weight of an oligopeptide from an amino acid single-letter sequence string.

Q16. Write an ANSI C program to reverse a string in-place using a recursive function without auxiliary string buffers.

Q17. Explain Function Pointers in C, detailing declaration syntax, typedef shortcuts, and callback dispatch tables.

Q18. Write an ANSI C program demonstrating the standard library qsort() with a custom comparator callback for floating-point gene scores.

Q19. Write an ANSI C program to compute the N-th Fibonacci number using Memoization to achieve O(N) linear time complexity.

Q20. Explain mutual recursion in C, demonstrating how forward function prototypes resolve circular dependencies between functions.

Q21. Explain reentrancy and thread-safety in C functions, detailing why internal static state renders functions non-reentrant.

Q22. Write an ANSI C program to recursively compute the length of a phylogenetic tree branch from a nested parent array representation.

Q23. Explain variable shadowing in C, detailing block scope nesting and potential masking bugs.

Q24. Write an ANSI C program to calculate the greatest common divisor (GCD) using tail recursion and compare with iterative execution.

Q25. Detail the atexit() function in C, explaining handler registration, execution order, and resource cleanup on exit.

Q26. Write an ANSI C program to search a sorted integer array using Recursive Binary Search, analyzing worst-case logarithmic complexity.

Q27. Explain the stack frame layout during nested and recursive function calls, detailing caller-saved vs callee-saved registers.

Q28. Write an ANSI C program to demonstrate a recursive power function x^n using binary exponentiation in O(log n) time.

Q29. Explain the dangers of returning pointers or references to local stack-allocated variables from C functions.

Q30. Write an ANSI C program to implement a modular codon translation table converting RNA triplets into amino acid names.

SECTION C: ANSWER KEY - MULTIPLE CHOICE QUESTIONS

Q1:	[A]	**Q21:**	[A]	**Q41:**	[A]	**Q61:**	[A]	**Q81:**	[A]
Q2:	[A]	**Q22:**	[A]	**Q42:**	[A]	**Q62:**	[A]	**Q82:**	[A]
Q3:	[A]	**Q23:**	[A]	**Q43:**	[A]	**Q63:**	[A]	**Q83:**	[A]
Q4:	[A]	**Q24:**	[A]	**Q44:**	[A]	**Q64:**	[A]	**Q84:**	[A]
Q5:	[A]	**Q25:**	[A]	**Q45:**	[A]	**Q65:**	[A]	**Q85:**	[A]
Q6:	[A]	**Q26:**	[A]	**Q46:**	[A]	**Q66:**	[A]	**Q86:**	[A]
Q7:	[A]	**Q27:**	[A]	**Q47:**	[A]	**Q67:**	[A]	**Q87:**	[A]
Q8:	[A]	**Q28:**	[A]	**Q48:**	[A]	**Q68:**	[A]	**Q88:**	[A]
Q9:	[A]	**Q29:**	[A]	**Q49:**	[A]	**Q69:**	[A]	**Q89:**	[A]
Q10:	[A]	**Q30:**	[A]	**Q50:**	[A]	**Q70:**	[A]	**Q90:**	[A]
Q11:	[A]	**Q31:**	[A]	**Q51:**	[A]	**Q71:**	[A]	**Q91:**	[A]
Q12:	[A]	**Q32:**	[A]	**Q52:**	[A]	**Q72:**	[A]	**Q92:**	[A]
Q13:	[A]	**Q33:**	[A]	**Q53:**	[A]	**Q73:**	[A]	**Q93:**	[A]
Q14:	[A]	**Q34:**	[A]	**Q54:**	[A]	**Q74:**	[A]	**Q94:**	[A]
Q15:	[A]	**Q35:**	[A]	**Q55:**	[A]	**Q75:**	[A]	**Q95:**	[A]
Q16:	[A]	**Q36:**	[A]	**Q56:**	[A]	**Q76:**	[A]	**Q96:**	[A]
Q17:	[A]	**Q37:**	[A]	**Q57:**	[A]	**Q77:**	[A]	**Q97:**	[A]
Q18:	[A]	**Q38:**	[A]	**Q58:**	[A]	**Q78:**	[A]	**Q98:**	[A]
Q19:	[A]	**Q39:**	[A]	**Q59:**	[A]	**Q79:**	[A]	**Q99:**	[A]
Q20:	[A]	**Q40:**	[A]	**Q60:**	[A]	**Q80:**	[A]	**Q100:**	[A]

MCQ Scoring Guide & Difficulty Distribution:

Questions 1 to 40: Foundational & Conceptual [Easy] | Questions 41 to 75: Analytical & Applied [Moderate]

Questions 76 to 100: Complex Numerical & Advanced Problem Solving [Difficult]

Anatomy of a C Function:
A C function consists of three distinct grammatical stages:
1. Function Prototype (Forward Declaration):
- Syntax: return_type function_name(parameter_type1 param1, parameter_type2 param2);
- Role: Informs compiler about the function's signature before its actual call site, allowing strict compile-time parameter type checking.
- Header Placement: Prototypes are placed in .h header files or at the top of .c files.

2. Function Definition:
- Composed of the Function Header followed by the Function Body { ... }:

```
return_type function_name(param_list) {
/* Local variable declarations */
/* Executable statements */
return result_value;
}
```

3. Function Call and Calling Conventions:
- Syntax: result = function_name(arg1, arg2);
- Calling Convention (e.g., cdecl on x86, System V AMD64 ABI on x86-64):
a. Caller evaluates actual arguments right-to-left and pushes them onto the stack or passes them in designated CPU registers (RDI, RSI, RDX, RCX, R8, R9).
b. Caller executes 'call' instruction, pushing the Return Address onto the stack.
c. Callee executes its body and places return value in RAX/EAX register.
d. Callee executes 'ret', popping return address back into instruction pointer (RIP/EIP).

4. Annotated ANSI C Code Listing:

```
#include <stdio.h>

/* 1. Function Prototype Declaration */
double compute_kinetic_velocity(double substrate_conc, double vmax, double km);

int main(void) {
double s = 2.5; /* Substrate concentration [S] in mM */
double vmax = 100.0; /* Vmax in umol/min */
double km = 1.2; /* Michaelis constant Km in mM */

/* 2. Function Invocation */
double v0 = compute_kinetic_velocity(s, vmax, km);
printf("Initial Reaction Velocity (v0): %.2f umol/min\n", v0);
return 0;
}

/* 3. Function Definition (Michaelis-Menten Enzyme Kinetics) */
double compute_kinetic_velocity(double substrate_conc, double vmax, double km) {
if (substrate_conc < 0.0 || (km + substrate_conc) <= 0.0) {
return 0.0;
}
/* v0 = (Vmax * [S]) / (Km + [S]) */
double velocity = (vmax * substrate_conc) / (km + substrate_conc);
return velocity; /* 4. Return statement yielding computed scalar */
}
```

Solution to Question 3: Compare Call-by-Value with Call-by-Reference in C, illustrating stack frame isolation and pointer-based parameter modification.

>>P Solution Statement Question 4: Explain array parameter passing in C, detailing array decay to pointers and why array length cannot be determined via sizeof inside fun

```
}
```

Solution to Question 5: Explain recursion in C, detailing base cases, recursive steps, call stack mechanics, and stack overflow conditions.

>>Solution to Question 6: Write an ANSI C program to solve the Tower of Hanoi problem recursively, detailing mathematical complexity and move proofs.

```
return 0;
}
```

>>Solution to Question 57: Explain Tail Recursion vs Non-Tail Recursion in C, detailing Tail Call Optimization (TCO) and stack frame recycling.

>>Solution Question Detail the default Storage Class of a static variable, external function in file C, architecture memory segment allocation.

- Declared inside a function block with the 'static' keyword.
- Initialized only ONCE at program startup.
- Unlike auto variables which are wiped upon function exit, local static variables retain their values across multiple calls:

```
#include <stdio.h>
int generate_sample_id(void) {
static int current_id = 1000; /* Stored in data segment, initialized once */
current_id++;
return current_id;
}
```

2. Global Static Variables (Internal Linkage):
- Declared outside of all functions with 'static'.
- Restricts symbol visibility STRICTLY TO THE CURRENT TRANSLATION UNIT (.c file).
- If file1.c declares static int config_mode = 1; and file2.c also declares static int config_mode = 2;, NO LINKER COLLISION OCCURS. Each file has its own private, isolated variable.

3. Global Static Functions (Private Helper Functions):
- In C, functions have external linkage by default.
- Marking a function static void parse_internal() limits its accessibility strictly to callers within the same .c file.
- Crucial Software Engineering Value:
Enforces encapsulation and information hiding. Internal helper functions cannot be called improperly by external modules, preventing unintended state mutations.

>> **Solution to Question 18:** external linkage using extern across multiple source files with an annotated multi-module C project layout.

```
#include "config.h"
int global_pcr_cycles = 35; /* Physical memory allocation */
void initialize_pcr(void) {
global_pcr_cycles = 40;
}

File 3: main.c (Client Module)
#include <stdio.h>
#include "config.h"
int main(void) {
printf("Initial PCR cycles: %d\n", global_pcr_cycles);
initialize_pcr();
printf("Updated PCR cycles: %d\n", global_pcr_cycles);
return 0;
}
Compilation Command: gcc config.c main.c -o pcr_sim.exe
```

>> Solution to Question 11: Explain inline functions in C99, comparing them with preprocessor macros and detailing compiler inlining heuristics.

Solution to Question 12: Implementing variadic functions in C using <stdarg.h>, implementing a custom formatting print function my_printf().

```
  default:
  putchar(*p);
  break;
  }
  }

  va_end(args);
  }

  int main(void) {
  mini_print("Gene: %s | Exons: %d | GC: %f%% | Class: %c\n", "BRCA2", 27, 41.5, 'A');
  return 0;
  }
```

>> Solution to Question 12: Write an ANSI C program to compute the Hamming Distance between two DNA oligonucleotides using an iterative function.

```
  int dist = compute_hamming_distance(wildtype, mutant);

  printf("=== Molecular Mutation Profiling ===\n");
  printf("Wildtype: %s\n", wildtype);
  printf("Mutant: %s\n", mutant);

  if (dist >= 0) {
  printf("Calculated Hamming Distance: %d point mutations\n", dist);
  printf("Homology Divergence: %.2f%%\n", ((double)dist / strlen(wildtype)) * 100.0);
  } else {
  printf("Error: Sequences differ in length!\n");
  }

  return 0;
  }
```

>>P Solution to Question 14: an ANSI C program implementing a modular function to transcribe a DNA coding sequence into messenger RNA (mRNA),

```
  printf("=== In-Vitro Molecular Transcription ===\n");
  printf("DNA Coding Strand: %s\n", dna_coding_strand);
  printf("mRNA Transcript: %s\n", mrna_transcript);

  return 0;
  }
```

>>Solution to Question 15: an ANSI C program to compute the molecular weight of an oligopeptide from an amino acid single-letter sequence string.

```
int main(void) {
const char *insulin_chain_a = "GIVEQCCTSICSLYQLENYCN";
double mw = compute_peptide_mw(insulin_chain_a);

printf("=== Peptide Mass Spectrometry Estimation ===\n");
printf("Sequence: %s\n", insulin_chain_a);
printf("Calculated Molecular Weight: %.2f Da (g/mol)\n", mw);

return 0;
}
```

>>Solution Question 16 Pointers to C-style strings: A pointer to a string literal (e.g., const char *) points to the first character of a null-terminated array. Functions like strlen() and strcmp() work by iterating through this array until they find the '\0' terminator.

[Detailed Step-by-Step Answer]:
Function Pointers and Callback Architecture in C:
1. Definition:
A function pointer stores the memory address of executable machine instructions for a function, enabling dynamic dispatch and callback routines.

2. Syntax Anatomy:
- Direct Declaration:
double (*calc_fp)(double, double);
Reads: calc_fp is a pointer to a function taking two doubles and returning double.
- Without Parentheses:
double *calc_fp(double, double);
Declares a function named calc_fp returning a pointer to double!

3. Simplification via typedef:
typedef double (*MathOperation)(double, double);
MathOperation op_ptr = add_numbers;

4. Callback Dispatch Table Code Example:

```
#include <stdio.h>

double add(double a, double b) { return a + b; }
double sub(double a, double b) { return a - b; }
double mul(double a, double b) { return a * b; }

typedef double (*BinaryFunc)(double, double);

int main(void) {
/* Array of function pointers (Dispatch Table) */
BinaryFunc dispatch_table[] = {add, sub, mul};
const char *names[] = {"Addition", "Subtraction", "Multiplication"};

double x = 20.0, y = 5.0;
for (int i = 0; i < 3; i++) {
double res = dispatch_table[i](x, y);
printf("Operation %-14s on (%.1f, %.1f) = %.2f\n", names[i], x, y, res);
}

return 0;
}
```

>> Solution to Question 18: Write an ANSI C program demonstrating the standard library qsort() with a custom comparator callback for floating-point gene scores.

2. Floating-Point Subtraction Trap:
Subtracting floats and casting to int ((int)(*f1 - *f2)) fails if difference is between 0.0 and 1.0 (truncates to 0)! Must use explicit comparison.

3. Complete ANSI C Source Code:

```
#include <stdio.h>
#include <stdlib.h>

typedef struct {
char gene_id[16];
double expression_score;
} GeneRecord;

/* Comparator for sorting descending by expression score */
int compare_genes_descending(const void *a, const void *b) {
const GeneRecord *g1 = (const GeneRecord *)a;
const GeneRecord *g2 = (const GeneRecord *)b;

if (g2->expression_score > g1->expression_score) return 1;
if (g2->expression_score < g1->expression_score) return -1;
return 0;
}

int main(void) {
GeneRecord dataset[] = {
{"BRCA1", 14.85},
{"TP53", 92.10},
{"MYC", 45.30},
{"EGFR", 78.60}
};
int count = sizeof(dataset) / sizeof(dataset[0]);

qsort(dataset, count, sizeof(GeneRecord), compare_genes_descending);

printf("=== Ranked Microarray Expression Profiles ===\n");
for (int i = 0; i < count; i++) {
printf("Rank %d: %-8s | Score: %6.2f FPKM\n",
i + 1, dataset[i].gene_id, dataset[i].expression_score);
}
return 0;
}
```

>> Solution to Question 19: Write an ANSI C program to compute the N-th Fibonacci number using Memoization to achieve O(N) linear time complexity.

Time complexity drops from O(2^N) to O(N).

2. Complete ANSI C Source Code:

```
#include <stdio.h>

#define MAX_FIB 90
unsigned long long fib_cache[MAX_FIB + 1];

unsigned long long fib_memo(int n) {
if (n <= 0) return 0ULL;
if (n == 1) return 1ULL;

/* Check lookup cache */
if (fib_cache[n] != 0) {
return fib_cache[n];
}

/* Compute and store */
fib_cache[n] = fib_memo(n - 1) + fib_memo(n - 2);
return fib_cache[n];
}

int main(void) {
printf("=== Memoized Top-Down Dynamic Programming ===\n");
for (int i = 1; i <= 50; i++) {
printf("Fib(%2d) = %llu\n", i, fib_memo(i));
}
return 0;
}
```

Solution to Question 20

>>> Explain mutual recursion in C, demonstrating how forward function prototypes resolve circular dependencies between functions.

```
bool is_even(unsigned int n) {
if (n == 0) return true; /* Base case: 0 is even */
return is_odd(n - 1); /* Mutual call to is_odd */
}

bool is_odd(unsigned int n) {
if (n == 0) return false; /* Base case: 0 is NOT odd */
return is_even(n - 1); /* Mutual call to is_even */
}

int main(void) {
unsigned int test_val = 14;
printf("Number %u is %s\n", test_val, is_even(test_val) ? "EVEN" : "ODD");
return 0;
}
```

>>Solution to Question 22: An ANSI C and thread-safe way to control the depth of white-list state branches for nested comment representation.

In computational evolutionary biology, phylogenetic trees represent genealogical ancestry.
A tree can be encoded as a parent array where parent[i] gives the ancestor node of node i (-1 denotes root).
Calculating distance from a node to the root measures evolutionary branch length.

2. Complete ANSI C Source Code:

```
#include <stdio.h>

int compute_branch_distance_recursive(const int *parent, int node) {
if (parent[node] == -1) {
return 0; /* Base case: Root node reached */
}
/* Recursive step: 1 + distance from parent to root */
return 1 + compute_branch_distance_recursive(parent, parent[node]);
}

int main(void) {
/* Tree structure: 0 (Root) -> 1, 2; 1 -> 3, 4; 4 -> 5 */
int parent_map[] = {-1, 0, 0, 1, 1, 4};
int total_nodes = sizeof(parent_map) / sizeof(parent_map[0]);

printf("=== Phylogenetic Branch Depth Analysis ===\n");
for (int i = 0; i < total_nodes; i++) {
int dist = compute_branch_distance_recursive(parent_map, i);
printf("Node %d: Evolutionary Distance to Root = %d steps\n", i, dist);
}
return 0;
}
```

>> Solution to Question 23: Explain variable shadowing in C, detailing block scope nesting and potential masking bugs.

```
printf("Level 1 (Post-Block): threshold = %d\n", threshold); /* Prints 50 */
}

int main(void) {
printf("Global Level: threshold = %d\n", threshold); /* Prints 100 */
inspect_shadowing();
return 0;
}
```

3. Hazards and Compiler Diagnostic Flags:
Shadowing often masks severe logical bugs where a developer accidentally re-declares a variable instead of modifying the outer variable.
Modern compilers flag this with: gcc -Wshadow.

>>P Solution to Question 24: an ANSI C program to calculate the greatest common divisor (GCD) using tail recursion and compare with iterative execution.

```
#include <stdio.h>

int binary_search_recursive(const int *arr, int low, int high, int target) {
if (low > high) return -1; /* Base case: Not found */

int mid = low + (high - low) / 2;

if (arr[mid] == target) {
return mid; /* Found target at index mid */
} else if (target < arr[mid]) {
return binary_search_recursive(arr, low, mid - 1, target);
} else {
return binary_search_recursive(arr, mid + 1, high, target);
}
}

int main(void) {
int sorted_gene_lengths[] = {120, 240, 380, 510, 720, 890, 1024, 2048};
int n = sizeof(sorted_gene_lengths) / sizeof(sorted_gene_lengths[0]);
int query = 720;

int idx = binary_search_recursive(sorted_gene_lengths, 0, n - 1, query);
if (idx != -1) {
printf("Gene length %d bp located at index: %d\n", query, idx);
} else {
printf("Gene length %d bp not present in database.\n", query);
}
return 0;
}
```

>>Solution to Question 27

Solution to Question 27

Explain the stack frame layout during nested and recursive function calls, detailing caller-saved vs callee-saved registers,

---

>>**Solution** **Question** **28** ANS: Cons of recursion: demonstrate a **recursive** over **function** **stack** **allocated** **variables** **from** **Citation**: O(log n) time.

```
return &local_val; /* COMPILER WARNING: Function returns address of local variable! */
}

void overwrite_stack(void) {
int dummy[16] = {999, 999, 999};
}

int main(void) {
int *ptr = faulty_allocator(); /* ptr now points to dead stack memory! */
printf("Dangling pointer accessed: %d\n", *ptr); /* May print 42 initially */
overwrite_stack(); /* Stack memory overwritten */
printf("After stack reuse: %d (CORRUPTED!)\n", *ptr); /* Garbage output! */
return 0;
}
```

3. Safe Solutions:
- Solution 1: Allocate dynamically on the heap: int *p = malloc(sizeof(int));
- Solution 2: Have caller allocate memory and pass the pointer into the function.
- Solution 3: Use static storage duration: static int val = 42;

>>Solution to Question 30: an ANSI C program to implement a modular codon translation table converting RNA triplets into amino acid names.

```
  for (int i = 0; i < count; i++) {
  printf("Codon %-4s translates to: %s\n", codons[i], translate_codon(codons[i]));
  }
  return 0;
  }
```
